



LLMServingSim 2.0 底层机制与时延模型

从 vLLM 风格调度到 Chakra / ASTRA-Sim 周期级仿真

汇报人

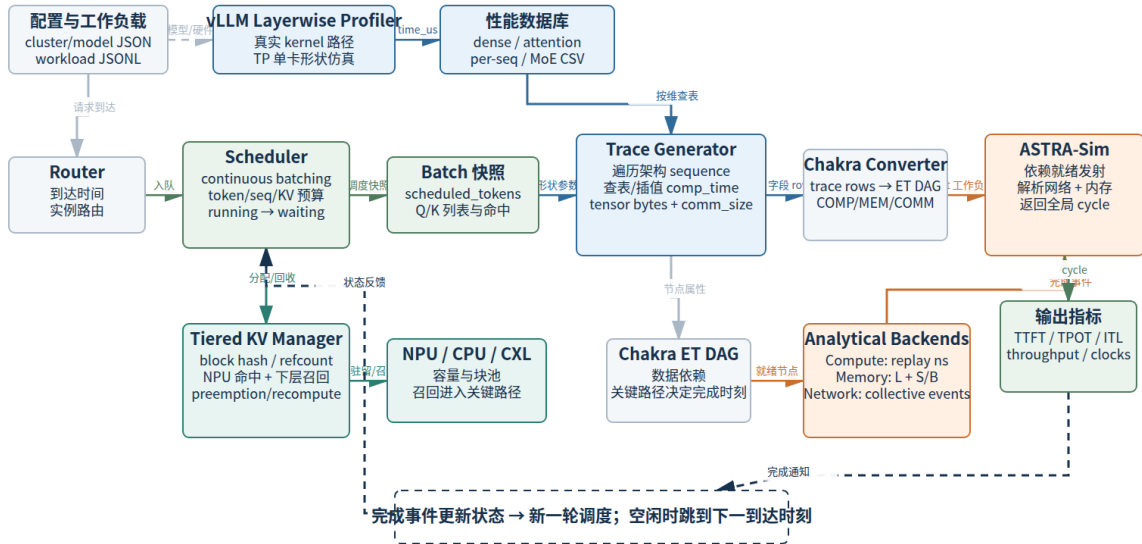
东南大学

2026 年 8 月 27 日

1. 为什么需要周期级模型
2. 系统执行机制
3. 计算时延模型
4. 访存与通信时延模型
5. 从节点到全局时钟

LLMServingSim 2.0: 周期级协同仿真的闭环机制

Python 前端复现服务语义, Chakra 表达依赖, ASTRA-Sim 推进硬件事件





仿真对象：一次迭代是一个可依赖的 DAG

前端状态

- Router: 按到达时间与实例负载分发
- Scheduler: running 优先, 受 token / seq / KV 预算约束
- Batch: 记录本轮实际 scheduled tokens 与 KV 命中
- Trace: 把 batch 形状映射到每层节点

后端事件

- COMP: 回放 profile 中的 kernel duration
- MEM: 按 memory tier 的带宽与延迟发事件
- COMM: 把 collective 拆成可调度的通信 phase
- 完成时刻: 所有依赖与资源约束同时满足

建模边界

调度、KV 与 trace 在 Python 中决定工作内容; Chakra 只表达依赖, ASTRA-Sim 推进硬件时钟。

`serving/core/scheduler.py:90--131; astra-sim/astra-sim/workload/Workload.cc:115--190`



统一输入：batch 形状，而不是“prefill/decode”标签

$$x = (T, N_{\text{seq}}, p_c, k_p, N_d, k_d^{\text{mean}}, k_d^{\text{max}}, TP, PP, EP)$$

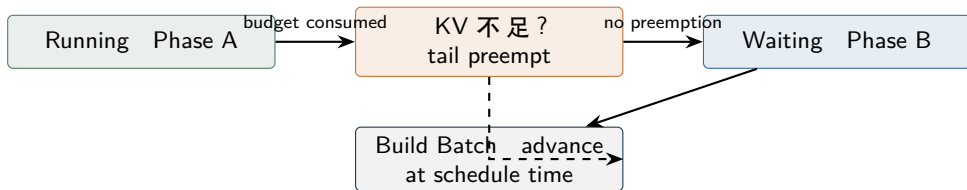
- $T = \sum_i n_i$: 本轮真正计算的 token 数
- $n_i > 1$: prefill chunk; $n_i = 1$: decode
- $p_c = \sum$ prefill query, $k_p = \sum$ prefill history
- N_d : decode request 数量
- $k_d^{\text{mean}} = \lfloor \sum k_i / N_d \rfloor$
- 非均匀 decode 额外使用 $k_d^{\text{max}}, k_d^{\text{min}}$

关键语义

恢复/重算仍按本轮 scheduled token 数分类；不能从 `num_computed_tokens` 推断序列长度。

`serving/core/scheduler.py:238--250, 272--338; serving/core/trace_generator.py:938--975`

调度：先服务 running，再尝试 admission



$$n_i^{\text{new}} = \min(\text{threshold}(L_i - \text{num_computed}_i), B_{\text{token}})$$

发生 preemption 的 step 跳过 waiting admission，避免“抢占 → 立即回填 → 再抢占”的震荡。

serving/core/scheduler.py:90--131, 133--236



KV 容量：静态权重与分页 KV 的分账

$$C_{KV} = \left\lfloor \frac{u M_{NPU} - W}{B_{block}} \right\rfloor B_{block}, \quad N_{block} = \left\lfloor \frac{u M_{NPU} - W}{B_{block}} \right\rfloor$$

- u : npu_memory_utilization; W : 单 NPU 保守权重上界
- $B_{block} = b_{tok} \cdot b_{KV/token/rank}$, 按 block 分配
- vLLM 会额外扣 activation peak / CUDA context; 本模拟器没有建模, 因此容量是同利用率下的上界

容量什么时候影响结果？

只有工作集触顶才会触发 preemption、重算或下层召回; 未触顶时容量不会直接改变 kernel duration。

`serving/core/memory_model.py:73--107; serving/core/kv_cache_manager.py:216--240`

KV 字节公式：GQA、层数、TP 与 FP8

$$H_{KV} = h_{KV} d_{\text{head}}, \quad b_{KV} = \begin{cases} 1, & \text{fp8 KV cache} \\ b_{\text{weight}}, & \text{otherwise} \end{cases}$$

$$B_{KV}(L, \text{per rank}) = \frac{2H_{KV} L N_{\text{layer}} b_{KV}}{N_{\text{NPU}}}$$

含义

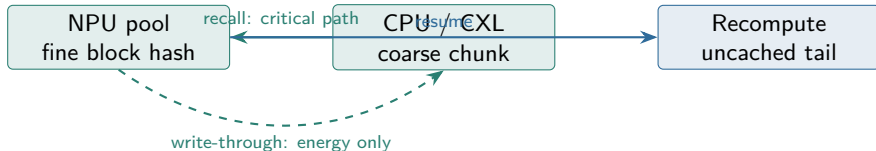
- 2: K 与 V
- H_{KV} : KV heads $\times$ head dimension
- N_{layer} : 每层都保存 KV

结果

- GQA 使 $h_{KV} < h_q$, KV 显著小于 QKV activation
- FP8 KV 直接将每元素从 2 bytes 降为 1 byte
- TP 只改变每 rank 持有的 shard

`serving/core/memory_model.py:48--55, 209--235`

层级 KV: 命中、召回与写回的时间语义



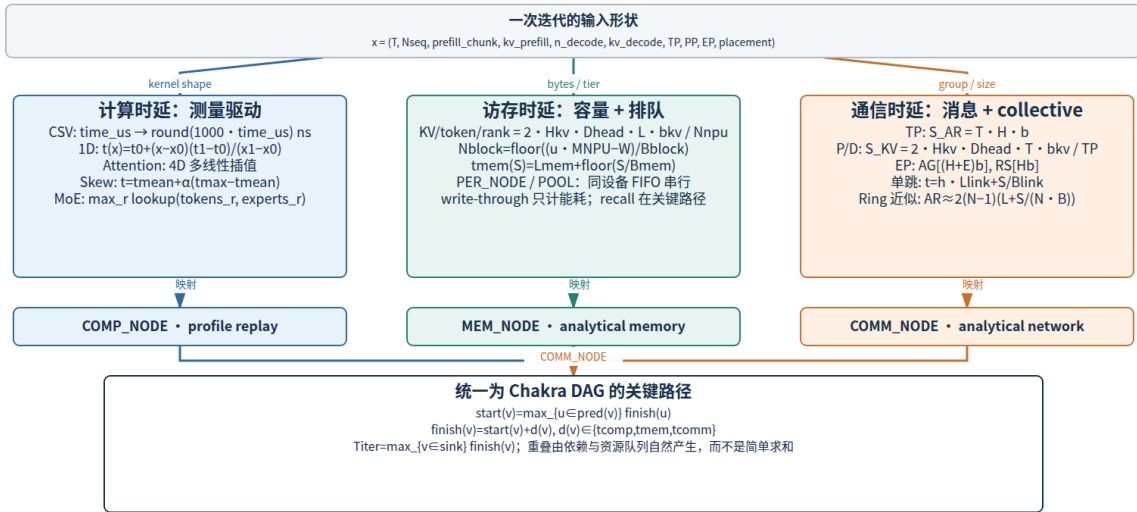
$$B_{\text{recall}} = N_{\text{coarse blocks}} \cdot B_{\text{coarse block}}, \quad B_{\text{write}} = N_{\text{newly full coarse blocks}} \cdot B_{\text{coarse block}}$$

free block 保留 hash; 因此 preemption 后可以恢复仍驻留的最长前缀。

serving/core/kv_cache_manager.py:128--195, 297--355

计算、访存与通信时延如何汇聚为一次迭代

测量驱动的计算节点 + 解析内存/网络节点 + DAG 关键路径





1D 计算查表: dense 与 per-sequence

$$t(x) = t_0 + \frac{x - x_0}{x_1 - x_0}(t_1 - t_0), \quad x \in \{T, N_{\text{seq}}\}$$

Dense layers

$x = T$; 典型层: embedding、qkv_proj、o_proj、FFN、norm。

CSV key: layer, tokens, time_us。

Per-sequence layers

$x = N_{\text{seq}}$; 典型层: lm_head、sampler。

CSV key: layer, sequences, time_us。

超过 profile 上界不截断: 使用最高两个点线性外推。

serving/core/trace_generator.py:440--468, 550--568; profiler/core/writer.py:173--209



Attention: 四维多线性插值

$$t_{\text{attn}} = \text{MLI}(p_c, k_p, N_d, k_d)$$

$$t_{00} = c_{00} + \lambda_d(c_{01} - c_{00}),$$

$$t_{10} = c_{10} + \lambda_d(c_{11} - c_{10}),$$

$$\boxed{t_{\text{attn}}} = t_{00} + \lambda_c(t_{10} - t_{00}).$$

- c_{ij} : 固定 (p_c, N_d) corner 内, 对 k_p 与 k_d 做两级线性插值
- 每个轴独立 bracket; 低于最小值 clamp, 高于最大值 extrapolate
- 运行时 decode 用 arithmetic mean: $N_d \bar{k}_d = \sum_i k_i$, 保持 KV read volume

`serving/core/trace_generator.py:571--639, 827--861`



Attention skew: 从均匀 anchor 修正 varlen 开销

$$t_{\text{mean}} = t(p_c, k_p, N_d, k_d^{\text{mean}}), \quad t_{\text{max}} = t(p_c, k_p, N_d, k_d^{\text{max}})$$

$$\boxed{t_{\text{skew}} = t_{\text{mean}} + \alpha (t_{\text{max}} - t_{\text{mean}})}, \quad \alpha^* = \frac{\sum_i \Delta t_{\text{mean},i} \Delta t_{\text{skew},i}}{\sum_i (\Delta t_{\text{mean},i})^2}$$

- α 按 $(p_c, N_d, \text{skew_rate}, k_d^{\text{max}}, k_p)$ 五轴 bucket 拟合
- $\text{skew_rate} = (k_d^{\text{mean}} - k_d^{\text{min}}) / (k_d^{\text{max}} - k_d^{\text{min}})$
- 无 profile、单 decode 或长度相等时, $\alpha = 0$, 只保留 t_{mean}

`profiler/core/fit_alpha.py:3--41, 268--350; serving/core/trace_generator.py:645--824`



MoE: 每个 EP rank 独立查表，慢者决定计算段

$$t_{\text{MoE}} = \max_{r \in \{1, \dots, EP\}} t_{\text{profile}}(T_r, E_r)$$

- router 产生每 rank 的 T_r 与 activated experts E_r
- moe.csv 在 TP=1 profile; 运行时按 EP rank token 数 lookup
- 专家权重按 EP shard, 而非 TP shard

serving/core/trace_generator.py:1105--1195; serving/core/memory_model.py:187--205

通信包络

dispatch: AllGather; expert compute;
combine: ReduceScatter。三段通过 DAG 依赖连接，不能把专家段平均化。



访存时延：解析公式 + 设备队列

$$t_{\text{mem}}(S) = L_{\text{mem}} + \left\lfloor \frac{S}{B_{\text{mem}}} \right\rfloor \text{ ns}$$

- `AnalyticalMemory::get_mem_runtime`: 对 tensor size S 加固定 latency, 再加带宽序列化时间
- `PER_NODE_MEMORY_EXPANSION` 与 `MEMORY_POOL`: 同 device 的请求进入 FIFO
- `PER_NPU_MEMORY_EXPANSION`: 每 NPU 独立注册事件, 体现更高并行度
- `mem-bw` 数值直接参与 S/B ; 网络后端另做 $\text{GB/s} \rightarrow \text{B/ns}$ 转换

建模结果

资源竞争改变的是事件开始时间; 节点自身的服务时间仍由 $L_{\text{mem}} + S/B_{\text{mem}}$ 给出。

`astra-sim/extern/memory_backend/analytical/AnalyticalMemory.cc:126--200, 271--275`



PIM attention: 可选的近存计算路径

$$B_{\text{ch}} = \frac{w_{\text{bus}}}{8} \frac{R_{\text{data}}}{1000} [\text{GB/s}], \quad L_{\text{read}} = CL \cdot t_{\text{CK}}$$

$$t_{\text{PIM}}(L) = \frac{a_0 r_{\text{GQA}} L + b_0 s_{\text{KV}}}{C_{\text{split}}}, \quad r_{\text{GQA}} = \frac{h_q/h_{\text{KV}}}{32/8}, \quad s_{\text{KV}} = \frac{h_q d_{\text{head}}}{32 \cdot 128}$$

- a_0, b_0 : 针对 LPDDR4X / DDR4 / LPDDR5 / HBM2 的基准拟合系数
- C_{split} : 同一 attention 在多个 PIM channel 上的并行切分数
- Chakra 生成 PIM_COMP_NODE; 内存后端将 PIM compute 与输入/输出 memory runtime 相加

`serving/core/pim_model.py:69--105, 120--185; AnalyticalMemory.cc:126--155`



从 layer shape 到 memory node: 哪些字节被发出去 ?

$$\begin{aligned}B_{\text{qkv,out}} &= T \frac{H_q + 2H_{\text{KV}}}{TP} b, \\B_{\text{attn,in}} &= T \frac{H_q}{TP} b + \frac{K_{\text{ctx}} H_{\text{KV}}}{TP} (2b), \\B_{\text{hidden}} &= THb.\end{aligned}$$

- calculate_sizes 为每个 canonical layer 产生 input / weight / output size
- attention 的输入同时包含 query 与 K/V context; $K_{\text{ctx}} = k_p + N_d k_d^{\text{mean}}$
- 首层 input 从 REMOTE 读, 最后 sampler output 写回 REMOTE; Chakra 生成 MEM LOAD/STORE

serving/core/memory_model.py:426--545; serving/core/trace_generator.py:993--1029;
astra-sim/extern/graph_frontend/chakra/src/converter/llm_converter.py:303--320



通信基础：单跳延迟是传播 + 序列化

$$B_{\text{link}} = B_{\text{GB/s}} \frac{2^{30}}{10^9} [\text{B/ns}]$$

$$t_{\text{hop}}(S, h) = hL_{\text{link}} + \frac{S}{B_{\text{link}}}$$

Congestion-unaware

- Ring: $h = \min(|dst - src|, N - |dst - src|)$
- FullyConnected: $h = 1$
- 直接返回解析 delay

Congestion-aware

- link busy 时排队
- 到达时间: $t + L + S/B$
- link 释放: $t + S/B$

`astra-sim/extern/network_backend/analytical/common/NetworkFunction.cpp:8--15;`
`congestion_unaware/basic-topology/BasicTopology.cpp:37--54; congestion_aware/network/Link.cpp:91--135`



TP、P/D、EP：前端如何确定通信大小？

$$S_{TP} = THb \quad \text{on } o_proj/down_proj \text{ ALLREDUCE}$$

$$S_{P/D} = \frac{2H_{KV} Tb_{KV}}{TP} \quad \text{per layer, per rank, K+V only}$$

$$S_{AG} = \max\left(1, \left\lfloor \frac{T}{EP} \right\rfloor\right) (H + E)b, \quad S_{RS} = THb$$

- P/D 不发送 Q：不能使用整个 qkv output，否则会按 $(H_q + 2H_{KV})/(2H_{KV})$ 高估
- EP dispatch/combine 是 AllGather / ReduceScatter； E 为 expert/router logits 相关维度

`serving/core/trace_generator.py:1047--1067, 1122--1148; serving/core/memory_model.py:226--235`

Ring collective: 通信量与 ASTRA-Sim 的 phase 分解

$$s = \frac{L}{N}, \quad D_{\text{AR}} = 2(N-1)s, \quad D_{\text{one pass}} = (N-1)s$$

$$T_{\text{AR}}^{\text{approx}} \approx 2(N-1) \left(L_{\text{link}} + \frac{L/N}{B_{\text{link}}} \right)$$

- AR: ReduceScatter + AllGather 两个 ring pass
- AG / RS / 某些 AllToAll: 单 pass 量级; 实际 phase 顺序由 collective implementation 与 topology 决定
- ASTRA-Sim 先按 preferred dataset split 切 chunk, 再沿 involved dimensions 生成 phases; phase 之间和多个 stream 会竞争队列

上式是解释通信量级的 ring 近似; 最终 cycle 来自 ASTRA-Sim 的事件调度。

`serving/core/power_model.py:191--207; astra-sim/astra-sim/system/Sys.cc:795--1005`



多维拓扑: `involved_dim` 决定谁参加 collective

DP + TP: $\mathbf{d} = [TP, DP]$, PP + DP + TP: $\mathbf{d} = [TP, PP, DP]$

- TP ALLREDUCE: $[1, 0]$ 或 $[1, 0, 0]$
- EP 跨 DP: $[1, 1]$ 或 $[1, 0, 1]$
- EP 不跨实例: 只激活 TP 维

为什么不能只写一个 N ?

同一条 trace 中, TP 与 EP 可能参与不同维度; 错误地把 PP 维加入 EP 会让不属于该 stage 的 NPU 进入 collective。

拓扑维度是性能模型的一部分, 不只是绘图布局。

`serving/core/config_builder.py:177--211, 719--737; astra-sim/astra-sim/workload/Workload.cc:306--340`



DAG 关键路径：不是简单把三种时延相加

$$start(v) = \max_{u \in pred(v)} finish(u), \quad finish(v) = start(v) + d(v)$$

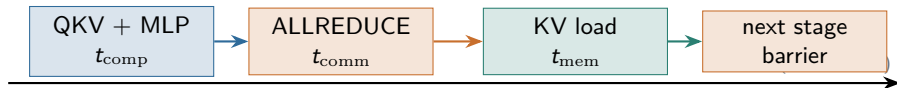
$$d(v) = \begin{cases} t_{profile}(x), & v = COMP \\ L_{mem} + S/B_{mem}, & v = MEM \\ T_{collective}(S, d, impl), & v = COMM \end{cases}$$

$$\boxed{T_{iter} = \max_{v \in sinks} finish(v)}, \quad T_{total} = \max_{r,j} finish(sink_{r,j})$$

依赖决定可见顺序；内存队列、链路占用、collective barrier 决定实际 start。

astra-sim/astra-sim/workload/Workload.cc:178--190, 306--355; serving/__main__.py:664--670, 1189--1199

一个 decode 迭代的瓶颈示例



KV load 与 compute 无数据依赖时可重叠；否则关键路径暴露访存时延

读图原则

先看依赖边，再看节点 duration，最后看共享资源队列；不要把 profile latency 当成整个迭代 latency。

`llm_converter.py:400--485; memory backend:158--200`



验证口径与适用边界

源码已核对

- 调度、KV、trace 与 Chakra 节点字段
- Analytical memory 的 $L + S/B$ 与 FIFO
- Analytical network 的 hop + serialization
- ring collective 的 chunk / phase 生成

需要环境验证

- 完整 ASTRA-Sim 二进制运行与 baseline clocks
- CUDA / vLLM profiler 的真实设备测量
- 非 ring collective 配置下的具体 phase 时间
- PIM 配置与不同设备的实际参数

解释边界

公式给出节点服务时间与通信量级；端到端结果仍由事件队列、数据依赖、collective barrier 与 workload 波次共同决定。

README.md; serving/README.md; AGENTS.md Testing and Validation

总结：三条公式串起整个仿真器

1. 计算: $t_{\text{comp}} = \text{profile lookup/interpolation}(T, N_{\text{seq}}, p_c, k_p, N_d, k_d)$ 。
2. 访存: $t_{\text{mem}} = L_{\text{mem}} + S/B_{\text{mem}}$, 容量决定是否发生 preemption / recall / recompute。
3. 通信: $t_{\text{link}} = hL_{\text{link}} + S/B_{\text{link}}$, collective 再按拓扑维度拆分 phase。

最终答案不是某一个公式，而是
“服务状态生成的 DAG，在硬件资源约束下的全局关键路径”。

参考文献



J. Cho, H. Choi, G. Heo, and J. Park.

LLMServingSim 2.0: A Unified Simulator for Heterogeneous and Disaggregated LLM Serving Infrastructure. ISPASS, 2026.



J. Cho, H. Choi, and J. Park.

LLMServingSim2.0: A Unified Simulator for Heterogeneous Hardware and Serving Techniques in LLM Infrastructure. IEEE CAL, 2025.



ASTRA-Sim project.

ASTRA-Sim: Analytical and Trace-based Network Simulation for Large-scale AI Systems.



Chakra project.

ET / Execution Trace Graph Frontend.



vLLM project.

vLLM V1 scheduler and paged KV cache design.

`README.md` Publications and Citation; `serving/README.md`



谢谢

Q&A

源码与公式备查



Backup: 层大小速查表

层	input / weight / output (per rank)	关键分片
qkv_proj	$THb / H(H_q + 2H_{KV})b / TP / T(H_q + 2H_{KV})b / TP$	TP
attention	$TH_qb / TP + K_{ctx}H_{KV}(2b) / TP / 0 / TH_qb / TP$	TP
o_proj	$T(H_q / TP)b / (H_q / TP)Hb / THb$	TP + AR
down_proj	$T(F / TP)b / (F / TP)Hb / THb$	TP + AR
moe	$THb / gate + E / EP \text{ experts} / THb$	EP + AG/RS
lm_head	$THb / H(V / TP)b / T(V / TP)b$	TP

serving/core/memory_model.py:461--540



Backup: P/D KV 传输为什么不能取 qkv output ?

$$B_{\text{qkv}} = T \frac{H_q + 2H_{\text{KV}}}{TP} b, \quad B_{\text{KV}} = T \frac{2H_{\text{KV}}}{TP} b_{\text{KV}}$$

$$\frac{B_{\text{qkv}}}{B_{\text{KV}}} = \frac{H_q + 2H_{\text{KV}}}{2H_{\text{KV}}} \cdot \frac{b}{b_{\text{KV}}}$$

- qkv projection 同时产生 Q、K、V；P/D paired decode 只需要 K、V
- Llama-3.1-8B 的 GQA 比例使旧写法约高估 3 倍，并且无法反映 FP8 KV
- 当前 trace 将 KV bytes 放入 qkv_proj 的 comm_size，converter 生成 SEND/RECV

`serving/core/trace_generator.py:1266--1280; serving/core/memory_model.py:226--235`



Backup: 关键源码导航

主题	入口
全局 handshake loop	serving/__main__.py:661--983
running/waiting 调度	serving/core/scheduler.py:90--236
trace 与查表	serving/core/trace_generator.py:550--861, 993--1195
KV 容量与层级	serving/core/memory_model.py:73--235
Chakra 节点	astra-sim/.../llm_converter.py:145--320
内存后端	astra-sim/.../AnalyticalMemory.cc:126--275
网络后端	astra-sim/.../BasicTopology.cpp;Link.cpp